

面向 Claude Code 的 Python 开源 CLI Agent 框架 Roadmap

一页概览：阶段目标 / 核心功能 / 完成标准

Phase 0: 项目初始化与技术基线 目标 建立可持续迭代的 Python 开源框架骨架，明确最小产品边界与工程规范。 核心功能 - 初始化 Python 项目结构 (src/tes - 建立 CLI 入口与基础命令框架 - 接入配置系统（环境变量、配置文件、命令行参数） - 建立日志系统与统一错误处理 - 建立测试框架、lint、format、type 完成标准 □ 可通过命令启动 CLI □ 本地开发、测试、格式化、静态检查流程跑通 □ 框架目录与核心抽象稳定	Phase 1: 最小 Agent Runtime 目标 实现 Claude Code 风格的最小执行闭环：用户输入、模型推理、工具调用、结果 核心功能 - 实现会话管理与消息历史记录 - 实现 Model Client 抽象，支持标准 - 实现 Agent Loop：推理 -> - 支持流式输出 - 支持 tool calling 完成标准 □ CLI 可完成多轮交互 □ 模型可以主动选择调用工具并基于结果继续执行 □ 一次请求可完成多步执行闭环	Phase 2: 核心本地工具集 目标 具备 Claude Code 最核心的本地工程操作能力，让 Agent 核心功能 - read_file - write_file - edit_file - list_files - search_code 完成标准 □ Agent 可读取、搜索、修改项目文件 □ Agent 可执行受控命令并消费命令输出 □ 所有工具具备统一注册、调用、返回结构	Phase 3: 代码仓上下文系统 目标 让 Agent 像 Claude Code 一样围绕代码仓上下文工作，而不是只依赖对话 核心功能 - Workspace 上下文收集 - Git 上下文收集 (branch、status - 项目规则文件加载（如 CLAUDE.md） - 当前任务相关文件聚合 - 上下文裁剪与优先级装配 完成标准 □ Agent 能自动感知当前仓库状态 □ Agent 能在回答和执行时使用项目约束 □ 上下文长度可控，不因大仓库直接失效
Phase 4: 任务规划与执行管理 目标 让 Agent 具备“先拆解任务，再逐步执行”的能力，形成 核心功能 - Task 数据结构与状态流转 (pending/ - Planner：从用户请求生成阶段任务 - 执行过程中的任务状态更新 - 任务依赖关系管理 - 失败任务重试与重新规划 完成标准 □ 复杂请求会先生成任务列表 □ Agent 在执行中能更新任务状态 □ 执行失败时可局部修正而不是整体失控	Phase 5: 权限、安全与可控自治 目标 建立 Claude Code 风格的人机协作安全机制，保证 Agent 核心功能 - 工具级风险分级 - 写文件、执行命令等操作的交互确认 - 路径级访问控制 - 命令黑白名单机制 - 危险操作拦截（删除、覆盖、外发） 完成标准 □ 高风险操作默认不可静默执行 □ Agent 的所有关键动作可追踪、可审计 □ 用户可配置权限策略	Phase 6: 开发工作流能力 目标 把 Agent 从“能调用工具”升级为“能参与真实开发流程”的工程助手。 核心功能 - Plan Mode - Review Mode - Fix Mode - Test Mode - Git 完成标准 □ Agent □ Agent 可在本地形成最基本的开发闭环 □ 常见研发操作具备统一入口和行为规范	Phase 7: Slash Commands / Skills 系统 目标 把高频流程沉淀为可复用能力，提升框架的开放性与可扩展性。 核心功能 - Slash Command 机制（如 - Skill 定义格式 - Skill 加载与发现机制 - Skill 级权限与上下文注入 - Skill 组合执行 完成标准 □ 高价值工作流可通过命令直接触发 □ 新能力可通过 Skill □ 框架具备开源生态扩展基础
Phase 8: Hooks 与自动化工作流 目标 让 Agent 能嵌入用户工程流程，形成 Claude Code 核心功能 - PreToolUse Hook - PostToolUse Hook - SessionEnd Hook - Hook 配置与执行上下文 - Hook 失败处理策略 完成标准 □ 工具执行前后可挂接自动化逻辑 □ 用户可用 Hook 定制团队工作流 □ 框架支持更强的工程自动化能力	Phase 9: 多 Agent 协作 目标 支持将复杂任务拆给不同角色 Agent 执行，提升复杂任务处理能力。 核心功能 - 子 Agent 生命周期管理 - Agent 角色定义 (planner/revi - 并行任务分发 - 子 Agent 结果汇总 - 主 Agent 调度与冲突控制 完成标准 □ 一个复杂任务可拆分给多个子 Agent □ 主 Agent 能汇总子 Agent □ 多 Agent 并行不会污染主会话状态	Phase 10: MCP 与外部能力接入 目标 建立标准化外部工具接入层，让框架具备平台化扩展能力。 核心功能 - MCP Client 抽象 - 外部资源与工具发现 - 外部文档、浏览器、数据库、Issue 系统接入 - 外部工具权限控制 - 本地工具与 MCP 工具统一调度 完成标准 □ 框架可接入外部 MCP 服务 □ 外部能力与本地工具在统一执行模型下协作 □ 插件生态边界清晰	Phase 11: 记忆、压缩与长会话能力 目标 提升长会话与持续协作能力，让 Agent 更接近真实长期开发助手。 核心功能 - Session 历史压缩 - Memory 抽象 (user/project/ - Memory 写入与检索策略 - 重要上下文摘要与回放 - 长任务恢复能力 完成标准 □ 长对话不会因上下文膨胀完全失效 □ Agent 能在后续会话中复用关键信息 □ 会话恢复与摘要机制稳定可用